

Lab Sheet 6: MongoDB Basic commands

Branch/ Class: B.tech

Faculty Name: Prof. S.Gopikrishnan

Student name: SIDDHI NIHARIKA

Date: 11-03-2026

School: SCOPE

Reg. no.: 23BCE9501

SLOT: L37+38

1. Use MongoDB to implement the following DB operations

1. Create a database called 'vehicles' and write a MongoDB query to select database as "vehicles".

```
test> use vehicles
switched to db vehicles
```

2. Write a MongoDB query to display all the databases.

```
vehicles> show dbs
Today          8.00 KiB
admin          48.00 KiB
config        108.00 KiB
jobPortal      24.00 KiB
local          88.00 KiB
school         56.00 KiB
```

3. Create a collection called 'two_wheelers'. (use capping) and Create a collection called 'four_wheelers'.

```
vehicles> db.createCollection("two_wheelers",{ capped:true,size:100000})
{ ok: 1 }
vehicles> db.createCollection("four_wheelers")
{ ok: 1 }
```

4. Add 5 two-wheeler details to the collection named 'two_wheelers'. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc, 125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

Input:

```
vehicles> db.two_wheelers.insertMany([
... {
...   bike_name: "Activa 6G",
...   model: "gearless",
...   category: "125cc",
...   colors_available: ["red","black","blue"],
...   manufacturer: "Honda",
...   performance: 8,
...   timestamp: new Date("2020-01-15"),
...   price: 90000
... },
... ])
```

```
... {
... bike_name: "Pulsar 150",
... model: "gear",
... category: "150cc",
... colors_available: ["black","sport red"],
... manufacturer: "Bajaj",
... performance: 9,
... timestamp: new Date("2019-03-10"),
... price: 110000
... },
...
... {
... bike_name: "Apache RTR 160",
... model: "gear",
... category: "160cc",
... colors_available: ["red","black"],
... manufacturer: "TVS",
... performance: 8,
... timestamp: new Date("2021-06-20"),
... price: 115000
... },
...
... {
... bike_name: "FZ-S",
... model: "gear",
... category: "150cc",
... colors_available: ["blue","black"],
... manufacturer: "Yamaha",
... performance: 7,
... timestamp: new Date("2020-05-11"),
... price: 120000
... },
...
... {
... bike_name: "Access 125",
... model: "gearless",
... category: "125cc",
... colors_available: ["white","black","blue"],
... manufacturer: "Suzuki",
... performance: 8,
... timestamp: new Date("2018-09-01"),
... price: 95000
... }
... ])
```

OUTPUT:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b13b092032003a6b50eb67'),
    '1': ObjectId('69b13b092032003a6b50eb68'),
    '2': ObjectId('69b13b092032003a6b50eb69'),
    '3': ObjectId('69b13b092032003a6b50eb6a'),
    '4': ObjectId('69b13b092032003a6b50eb6b')
  }
}
```

5. Add 5 four-wheeler details to the collection named 'four_wheelers'. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

Input:

```
vehicles> db.four_wheelers.insertMany([
... {
...   vehicle_name: "Swift",
...   model: "own",
...   category: "car",
...   variants: ["vxi","zxi","petrol"],
...   manufacturer: "Maruti Suzuki",
...   performance: 8,
...   timestamp: new Date("2020-02-10"),
...   price: 700000
... },
...
... {
...   vehicle_name: "Innova",
...   model: "own",
...   category: "car",
...   variants: ["diesel","zxi"],
...   manufacturer: "Toyota",
...   performance: 9,
...   timestamp: new Date("2019-11-20"),
...   price: 1800000
... },
...
... {
...   vehicle_name: "Ashok Leyland Dost",
...   model: "commercial",
...   category: "mini truck",
...   variants: ["diesel"],
...   manufacturer: "Ashok Leyland",
...   performance: 7,
...   timestamp: new Date("2018-07-15"),
```

```

... price: 900000
... },
...
... {
... vehicle_name: "Tata Lorry",
... model: "commercial",
... category: "heavy truck",
... variants: ["diesel"],
... manufacturer: "Tata",
... performance: 6,
... timestamp: new Date("2017-03-25"),
... price: 2500000
... },
...
... {
... vehicle_name: "Volvo Bus",
... model: "commercial",
... category: "bus",
... variants: ["diesel"],
... manufacturer: "Volvo",
... performance: 9,
... timestamp: new Date("2021-01-10"),
... price: 5000000
... }
... ]

```

Output:

```

{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b13bd32032003a6b50eb6c'),
    '1': ObjectId('69b13bd32032003a6b50eb6d'),
    '2': ObjectId('69b13bd32032003a6b50eb6e'),
    '3': ObjectId('69b13bd32032003a6b50eb6f'),
    '4': ObjectId('69b13bd32032003a6b50eb70')
  }
}

```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

Input:

```
vehicles> db.two_wheelers.find()
```

```
db.four_wheelers.find()
```

output:

```
[
  {
    _id: ObjectId('69b13b092032003a6b50eb67'),
    bike_name: 'Activa 6G',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'red', 'black', 'blue' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: ISODate('2020-01-15T00:00:00.000Z'),
    price: 90000
  },
  {
    _id: ObjectId('69b13b092032003a6b50eb68'),
    bike_name: 'Pulsar 150',
    model: 'gear',
    category: '150cc',
    colors_available: [ 'black', 'sport red' ],
    manufacturer: 'Bajaj',
    performance: 9,
    timestamp: ISODate('2019-03-10T00:00:00.000Z'),
    price: 110000
  },
  {
    _id: ObjectId('69b13b092032003a6b50eb69'),
    bike_name: 'Apache RTR 160',
    model: 'gear',
    category: '160cc',
    colors_available: [ 'red', 'black' ],
    manufacturer: 'TVS',
    performance: 8,
    timestamp: ISODate('2021-06-20T00:00:00.000Z'),
    price: 115000
  },
  {
    _id: ObjectId('69b13b092032003a6b50eb6a'),
    bike_name: 'FZ-S',
```

```

vehicles> db.four_wheelers.find()
[
  {
    _id: ObjectId('69b13bd32032003a6b50eb6c'),
    vehicle_name: 'Swift',
    model: 'own',
    category: 'car',
    variants: [ 'vxi', 'zxi', 'petrol' ],
    manufacturer: 'Maruti Suzuki',
    performance: 8,
    timestamp: ISODate('2020-02-10T00:00:00.000Z'),
    price: 700000
  },
  {
    _id: ObjectId('69b13bd32032003a6b50eb6d'),
    vehicle_name: 'Innova',
    model: 'own',
    category: 'car',
    variants: [ 'diesel', 'zxi' ],
    manufacturer: 'Toyota',
    performance: 9,
    timestamp: ISODate('2019-11-20T00:00:00.000Z'),
    price: 1800000
  },
  {
    _id: ObjectId('69b13bd32032003a6b50eb6e'),
    vehicle_name: 'Ashok Leyland Dost',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 7,
    timestamp: ISODate('2018-07-15T00:00:00.000Z'),
    price: 900000
  }
]

```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database

input:

```
db.two_wheelers.find({}, {bike_name:1, price:1, _id:0})
```

```
db.four_wheelers.find({}, {vehicle_name:1, price:1, _id:0})
```

output:

```

vehicles> db.two_wheelers.find({}, {bike_name:1, price:1, _id:0})
[
  { bike_name: 'Activa 6G', price: 90000 },
  { bike_name: 'Pulsar 150', price: 110000 },
  { bike_name: 'Apache RTR 160', price: 115000 },
  { bike_name: 'FZ-S', price: 120000 },
  { bike_name: 'Access 125', price: 95000 }
]
vehicles> db.four_wheelers.find({}, {vehicle_name:1, price:1, _id:0})
[
  { vehicle_name: 'Swift', price: 700000 },
  { vehicle_name: 'Innova', price: 1800000 },
  { vehicle_name: 'Ashok Leyland Dost', price: 900000 },
  { vehicle_name: 'Tata Lorry', price: 2500000 },
  { vehicle_name: 'Volvo Bus', price: 5000000 }
]

```

8. Write a MongoDB query to display two_wheelers from a particular company

Input:

```
db.two_wheelers.find({manufacturer:"Honda"})
```

output:

```
vehicles> db.two_wheelers.find({manufacturer:"Honda"})
[
  {
    _id: ObjectId('69b13b092032003a6b50eb67'),
    bike_name: 'Activa 6G',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'red', 'black', 'blue' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: ISODate('2020-01-15T00:00:00.000Z'),
    price: 90000
  }
]
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

Input:

```
db.four_wheelers.find({variants:"diesel"})
```

output:

```
vehicles> db.four_wheelers.find({variants:"diesel"})
[
  {
    _id: ObjectId('69b13bd32032003a6b50eb6d'),
    vehicle_name: 'Innova',
    model: 'own',
    category: 'car',
    variants: [ 'diesel', 'zxi' ],
    manufacturer: 'Toyota',
    performance: 9,
    timestamp: ISODate('2019-11-20T00:00:00.000Z'),
    price: 1800000
  },
  {
    _id: ObjectId('69b13bd32032003a6b50eb6e'),
    vehicle_name: 'Ashok Leyland Dost',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 7,
    timestamp: ISODate('2018-07-15T00:00:00.000Z'),
    price: 900000
  },
  {
    _id: ObjectId('69b13bd32032003a6b50eb6f'),
    vehicle_name: 'Tata Lorry',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel' ],
    manufacturer: 'Tata',
    performance: 6,
    timestamp: ISODate('2017-03-25T00:00:00.000Z'),
    price: 2500000
  }
]
```


10. Write a MongoDB query to display vehicles name, category

Input:

```
db.two_wheelers.find(
... {performance:{$gt:5}},
... {bike_name:1, category:1, manufacturer:1, _id:0}
... )
```

Output:

```
vehicles> db.two_wheelers.find(
... {performance:{$gt:5}},
... {bike_name:1, category:1, manufacturer:1, _id:0}
... )
[
  { bike_name: 'Activa 6G', category: '125cc', manufacturer: 'Honda' },
  { bike_name: 'Pulsar 150', category: '150cc', manufacturer: 'Bajaj' },
  {
    bike_name: 'Apache RTR 160',
    category: '160cc',
    manufacturer: 'TVS'
  },
  { bike_name: 'FZ-S', category: '150cc', manufacturer: 'Yamaha' },
  {
    bike_name: 'Access 125',
    category: '125cc',
    manufacturer: 'Suzuki'
  }
]
```

Input:

```
db.four_wheelers.find(
... {performance:{$gt:5}},
... {vehicle_name:1, category:1, manufacturer:1, _id:0}
... )
```

Output:

```
vehicles> db.four_wheelers.find(
... {performance:{$gt:5}},
... {vehicle_name:1, category:1, manufacturer:1, _id:0}
... )
[
  {
    vehicle_name: 'Swift',
    category: 'car',
    manufacturer: 'Maruti Suzuki'
  },
  {
    vehicle_name: 'Ashok Leyland Dost',
    category: 'mini truck',
    manufacturer: 'Ashok Leyland'
  },
  {
    vehicle_name: 'Tata Lorry',
    category: 'heavy truck',
    manufacturer: 'Tata'
  },
  { vehicle_name: 'Volvo Bus', category: 'bus', manufacturer: 'Volvo' }
]
vehicles> █
```


2. Use MongoDB to implement the following DB operations for a Zoo
1. Create a database called 'animal' and write a MongoDB query to select database as 'animal'.

```
test> use animal
switched to db animal
```

2. Write a MongoDB query to display all the databases.

```
animal> show dbs
Today          8.00 KiB
admin          48.00 KiB
config         72.00 KiB
jobPortal      24.00 KiB
local          88.00 KiB
school         56.00 KiB
vehicles       80.00 KiB
```

3. Create a collection called 'wild_animals'.(use capping) and Create a collection called 'domestic_animals'.

Input:

```
animal> db.createCollection("wild_animals", { capped: true, size: 100000 })
animal> db.createCollection("domestic_animals")
```

output:

```
animal> db.createCollection("wild_animals", { capped: true, size: 100000
  })
{ ok: 1 }
animal> db.createCollection("domestic_animals")
{ ok: 1 }
animal> db.wild_animals.insertMany([
```

4. Add 5 wild_animal details to the collection named 'wild_animals'. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

Input:

```
db.wild_animals.insertMany([

... {
... animal_name: "Lion",
... nature: "harm",
... favorite_foods: ["meat","deer"],

... care_taker_name: "Ravi",
... life_span: 14,
... timestamp: new Date("2020-01-10"),
... expenses: 50000
... },
...
... {
... animal_name: "Tiger",
```

```

... nature: "harm",
... favorite_foods: ["meat","rabbit"],
... care_taker_name: "Suresh",
... life_span: 16,
... timestamp: new Date("2019-05-20"),
... expenses: 60000
... },
...
... {
... animal_name: "Elephant",
... nature: "harmless",
... favorite_foods: ["grass","fruits"],
... care_taker_name: "Mahesh",
... life_span: 60,
... timestamp: new Date("2018-08-15"),
... expenses: 70000
... },
...
... {
... animal_name: "Leopard",
... nature: "harm",
... favorite_foods: ["meat","deer"],
... care_taker_name: "Ravi",
... life_span: 12,
... timestamp: new Date("2021-03-05"),
... expenses: 45000
... },
...
... {
... animal_name: "Deer",
... nature: "harmless",
... favorite_foods: ["grass","plants"],
... care_taker_name: "Anil",
... life_span: 10,
... timestamp: new Date("2022-06-11"),
... expenses: 20000
... }
... ])

```

Output:

```

{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b156b80a42f9d0ea50eb67'),
    '1': ObjectId('69b156b80a42f9d0ea50eb68'),
    '2': ObjectId('69b156b80a42f9d0ea50eb69'),
    '3': ObjectId('69b156b80a42f9d0ea50eb6a'),
    '4': ObjectId('69b156b80a42f9d0ea50eb6b')
  }
}

```

5. Add 5 domestic-animal details to the collection named 'domestic_animals'. Each document consists of following fields as animal_name, gender (male or female), favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

Input:

```
db.domestic_animals.insertMany([
... {
... animal_name: "Dog",
... gender: "male",
... favorite_foods: ["meat","rice"],

... animal_petname: "Rocky",
... life_span: 12,
... timestamp: new Date("2021-02-10"),
... expenses: 15000
... },
...
... {
... animal_name: "Cat",
... gender: "female",
... favorite_foods: ["milk","fish"],

... animal_petname: "Kitty",
... life_span: 14,
... timestamp: new Date("2020-07-15"),
... expenses: 10000
... },
...
... {
... animal_name: "Cow",
... gender: "female",
... favorite_foods: ["grass","plants"],
... animal_petname: "Gauri",
... life_span: 20,
... timestamp: new Date("2019-03-18"),
... expenses: 25000
... },
...
... {
... animal_name: "Goat",
... gender: "male",
... favorite_foods: ["grass","leaves"],
... animal_petname: "Bunny",
... life_span: 15,
... timestamp: new Date("2022-01-11"),
... expenses: 8000
```

```

... },
...
... {
... animal_name: "Horse",
... gender: "male",
... favorite_foods: ["grass","grains"],
... animal_petname: "Thunder",
... life_span: 25,
... timestamp: new Date("2018-09-05"),
... expenses: 30000
... }
... ])

```

Output:

```

{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b156c20a42f9d0ea50eb6c'),
    '1': ObjectId('69b156c20a42f9d0ea50eb6d'),
    '2': ObjectId('69b156c20a42f9d0ea50eb6e'),
    '3': ObjectId('69b156c20a42f9d0ea50eb6f'),
    '4': ObjectId('69b156c20a42f9d0ea50eb70')
  }
}

```

6. Write a MongoDB query to display all documents available in wild_animals and domestic_animals.

Input: db.wild_animals.find()

Output:

```

animal> db.wild_animals.find()
[
  {
    _id: ObjectId('69b156b80a42f9d0ea50eb67'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 14,
    timestamp: ISODate('2020-01-10T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b156b80a42f9d0ea50eb68'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbit' ],
    care_taker_name: 'Suresh',
    life_span: 16,
    timestamp: ISODate('2019-05-20T00:00:00.000Z'),
    expenses: 60000
  },
  {
    _id: ObjectId('69b156b80a42f9d0ea50eb69'),
    animal_name: 'Elephant',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'fruits' ],
    care_taker_name: 'Mahesh',
    life_span: 60,
    timestamp: ISODate('2018-08-15T00:00:00.000Z'),
    expenses: 70000
  },
]

```

Input: db.domestic_animals.find()

Output:

```
animal> db.domestic_animals.find()
[
  {
    _id: ObjectId('69b156c20a42f9d0ea50eb6c'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [ 'meat', 'rice' ],
    animal_petname: 'Rocky',
    life_span: 12,
    timestamp: ISODate('2021-02-10T00:00:00.000Z'),
    expenses: 15000
  },
  {
    _id: ObjectId('69b156c20a42f9d0ea50eb6d'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'milk', 'fish' ],
    animal_petname: 'Kitty',
    life_span: 14,
    timestamp: ISODate('2020-07-15T00:00:00.000Z'),
    expenses: 10000
  },
  {
    _id: ObjectId('69b156c20a42f9d0ea50eb6e'),
    animal_name: 'Cow',
    gender: 'female',
    favorite_foods: [ 'grass', 'plants' ],
    animal_petname: 'Gauri',
    life_span: 20,
    timestamp: ISODate('2019-03-18T00:00:00.000Z'),
    expenses: 25000
  },
]
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

input:

db.wild_animals.find({}, {animal_name:1, expenses:1, _id:0})

db.domestic_animals.find({}, {animal_name:1, expenses:1, _id:0})

output:

```
animal> db.wild_animals.find({}, {animal_name:1, expenses:1, _id:0})
[
  { animal_name: 'Lion', expenses: 50000 },
  { animal_name: 'Tiger', expenses: 60000 },
  { animal_name: 'Elephant', expenses: 70000 },
  { animal_name: 'Leopard', expenses: 45000 },
  { animal_name: 'Deer', expenses: 20000 }
]
animal> db.domestic_animals.find({}, {animal_name:1, expenses:1, _id:0})
[
  { animal_name: 'Dog', expenses: 15000 },
  { animal_name: 'Cat', expenses: 10000 },
  { animal_name: 'Cow', expenses: 25000 },
  { animal_name: 'Goat', expenses: 8000 },
  { animal_name: 'Horse', expenses: 30000 }
]
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

Input:

```
db.domestic_animals.find({life_span:12})
```

output:

```
animal> db.domestic_animals.find({life_span:12})
[
  {
    _id: ObjectId('69b156c20a42f9d0ea50eb6c'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [ 'meat', 'rice' ],
    animal_petname: 'Rocky',
    life_span: 12,
    timestamp: ISODate('2021-02-10T00:00:00.000Z'),
    expenses: 15000
  }
]
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker

input:

```
db.wild_animals.find({care_taker_name:"Ravi"})
```

output:

```
animal> db.wild_animals.find({care_taker_name:"Ravi"})
[
  {
    _id: ObjectId('69b156b80a42f9d0ea50eb67'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 14,
    timestamp: ISODate('2020-01-10T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b156b80a42f9d0ea50eb6a'),
    animal_name: 'Leopard',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 12,
    timestamp: ISODate('2021-03-05T00:00:00.000Z'),
    expenses: 45000
  }
]
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

Input:

```
db.wild_animals.find(  
... {life_span:{$gt:5}},  
... {animal_name:1, favorite_foods:1, expenses:1, _id:0}  
... )
```

Output:

```
animal> db.wild_animals.find(  
... {life_span:{$gt:5}},  
... {animal_name:1, favorite_foods:1, expenses:1, _id:0}  
... )  
[  
  {  
    animal_name: 'Lion',  
    favorite_foods: [ 'meat', 'deer' ],  
    expenses: 50000  
  },  
  {  
    animal_name: 'Tiger',  
    favorite_foods: [ 'meat', 'rabbit' ],  
    expenses: 60000  
  },  
  {  
    animal_name: 'Elephant',  
    favorite_foods: [ 'grass', 'fruits' ],  
    expenses: 70000  
  },  
  {  
    animal_name: 'Leopard',  
    favorite_foods: [ 'meat', 'deer' ],  
    expenses: 45000  
  },  
]
```


Input:

```
db.domestic_animals.find(
... {life_span:{$gt:5}},
... {animal_name:1, favorite_foods:1, expenses:1, _id:0}
... )
```

Output:

```
animal> db.domestic_animals.find(
... {life_span:{$gt:5}},
... {animal_name:1, favorite_foods:1, expenses:1, _id:0}
... )
[
  {
    animal_name: 'Dog',
    favorite_foods: [ 'meat', 'rice' ],
    expenses: 15000
  },
  {
    animal_name: 'Cat',
    favorite_foods: [ 'milk', 'fish' ],
    expenses: 10000
  },
  {
    animal_name: 'Cow',
    favorite_foods: [ 'grass', 'plants' ],
    expenses: 25000
  },
  {
    animal_name: 'Goat',
    favorite_foods: [ 'grass', 'leaves' ],
    expenses: 8000
  },
  {
    animal_name: 'Horse',
    favorite_foods: [ 'grass', 'grains' ],
    expenses: 30000
  }
]
```